

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I k.Divya (192511316) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.Divya

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

Legacy E-Learning Management System

The legacy E-Learning Management System combines student registration, course management, content delivery, assignment evaluation, result generation, UI logic, and business rules in large modules. This creates low cohesion and strong coupling: a change to a business rule can affect screens, persistence, and unrelated features.

Reverse-engineered domain classes

Class	Key attributes	Responsibilities / methods
Student	studentId, name, email, status	register(), enroll(), submitAssignment()
Course	courseId, title, description, capacity	addContent(), addAssignment(), enrollStudent()
Enrollment	enrollmentId, enrolledAt, status	activate(), cancel()
CourseContent	contentId, title, type, location	publish(), access()
Assignment	assignmentId, title, dueDate, maxMarks	submit(), validateDueDate()
Submission	submissionId, submittedAt, fileRef, status	submit(), markLate()
Evaluation	evaluationId, marks, feedback	evaluate(), calculateGrade()
Result	resultId, totalMarks, grade, publishedAt	calculate(), publish()

Layered redesign

Presentation Layer — Student UI, Instructor UI, Admin UI, controllers, and DTOs.

Application Layer — Student Service, Course Service, Assignment Service, Result Service; coordinates use cases.

Domain Layer — Student, Course, Enrolment, Assignment, Submission, Evaluation, Result and business rules.

Persistence Layer — Student Repository, Course Repository, Enrolment Repository, Submission Repository, Result Repository.

Infrastructure Layer — database adapters, file/content storage, authentication, and external integrations.

Relationships

Student 1..* Enrolment *..1 Course. Enrolment models the student-course association.

Course 1..* Course Content and 1.* Assignment.

Assignment 1..* Submission; each Submission belongs to a student.

Submission 0..1 Evaluation; evaluations contribute to a Result.

Application services depend on repository interfaces, not concrete database classes.

Design improvement

Cohesion increases because each class has one focused responsibility. Coupling decreases because presentation and domain logic communicate through services and DTOs, while persistence is hidden behind repositories. New delivery formats, evaluation rules, or content stores can be introduced without rewriting the entire application, improving maintainability and scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

Legacy Customer Support System

The legacy support application uses repeated conditional statements to select email, chatbot, telephone, or social-media behavior. This is a classic sign of rigid conditional dispatch: every new channel forces modifications to existing modules and mixes channel-specific behavior with routing and business logic.

Object-oriented limitations

Large if/else or switch blocks violate the Open/Closed Principle.

Existing modules know every concrete support channel.
Channel behavior, validation, routing, logging, and escalation become mixed.
Adding a channel increases regression risk and requires broader testing.
Concrete dependencies make unit testing and replacement difficult.

Reverse-engineered classes

Class / interface	Key attributes	Methods
SupportRequest	requestId, customerId, message, priority, channel, status	validate(), updateStatus()
SupportChannel	abstract channel	send(), receive(), validate()
EmailChannel	email configuration	send(), receive()
ChatbotChannel	bot configuration	send(), receive()
TelephoneChannel	call configuration	send(), receive()
SocialMediaChannel	platform configuration	send(), receive()
SupportRouter	handler/strategy chain	route(), escalate()

Behavioural-pattern redesign

Strategy Pattern — Support Channel is an interface; each channel implements the same contract. The selected strategy performs channel-specific behaviour.
Chain of Responsibility — routing, validation, priority handling, escalation, and specialist handlers can process a request sequentially.
Observer / Publisher-Subscriber — optional event publication can notify dashboards, audit logs, or monitoring services without coupling them to support channels.

Result

Adding WhatsApp Channel or another future channel becomes an additive change: implement the interface and register it. Existing channel classes and routing logic remain stable. This provides flexibility, extensibility, loose coupling, and easier testing.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

Legacy Banking Transaction System

The legacy banking system combines account management, validation, transfers, loans, reporting, and database operations inside a few tightly coupled classes. Business methods directly access the database, making domain rules dependent on infrastructure and making multi-step financial operations harder to test and maintain.

Domain classes

Class	Key attributes	Responsibilities / methods
Customer	customerId, name, contactDetails	updateProfile()
Account	accountId, accountNumber, balance, status, accountType	credit(), debit(), freeze(), activate()
Transaction	transactionId, type, amount, timestamp, status	validate(), complete(), fail()
Transfer	transferId, sourceAccount, destinationAccount, amount	validate(), execute()
Loan	loanId, principal, interestRate, term, status	approve(), calculateRepayment(), disburse()
TransactionReport	period, transactions, totals	generate(), export()

Services and data-access components

AccountService — account lifecycle and account use cases.

TransactionService — transaction creation, validation, and status management.

TransferService — coordinates source/destination validation, debit, credit, and transaction recording.

LoanService — eligibility, approval, repayment calculations, and disbursement workflow.

ReportingService — obtains data through repository interfaces and produces reports.

AccountRepository, TransactionRepository, LoanRepository, CustomerRepository — persistence abstractions.

UnitOfWork / TransactionManager — coordinates atomic persistence for multi-step financial operations.

Architecture and patterns

Presentation/API → Application Services → Domain Model → Repository Interfaces → Infrastructure/Database.

Repository Pattern isolates database details.

Dependency Injection supplies repository implementations to services.

Strategy can encapsulate variable validation, fee, or interest-calculation rules.

Factory Method can create transaction objects according to transaction type.

Result

Business rules become independent of SQL or OODBMS APIs. Services can use mocks or in-memory repositories in unit tests. Database technology can change with limited impact, while explicit transaction boundaries improve reliability and maintainability.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

Legacy Hotel Reservation System

The reservation screens directly perform OODBMS operations. Queries for rooms, guests, bookings, and payments are repeated in several modules. This creates presentation-to-database coupling, duplicated persistence logic, inconsistent data access, and difficult OODBMS integration.

Reverse-engineered domain model

Class	Key attributes	Responsibilities / methods
Hotel	hotelId, name, location	addRoom(), findRoom()
Guest	guestId, name, contactDetails, preferences	register(), updateDetails()
Room	roomId, roomNumber, roomType, status, rate	isAvailable(), reserve(), release()
Reservation	reservationId, guest, room, checkIn, checkOut, status	confirm(), cancel(), modify()
Payment	paymentId, reservationId, amount, method, status	authorize(), capture(), refund()

DAO / Repository redesign

RoomDAO / RoomRepository — findAvailableRooms(), findById(), save(), updateStatus().

GuestDAO / GuestRepository — findById(), findByContact(), save(), update().

ReservationRepository — findById(), findByGuest(), findOverlapping(), save(), cancel().

PaymentRepository — findByReservation(), save(), updateStatus().

ReservationService — coordinates availability rules, reservation lifecycle, payment, and persistence.

DAO can expose OODBMS-oriented operations; Repository can expose domain/aggregate-oriented collections. Both can coexist behind service abstractions.

OODBMS integration and benefits

Infrastructure implementations translate repository operations into OODBMS queries or object navigation.

Object identity, relationship loading, caching, connection management, and transactions are centralized.

Repeated queries disappear from presentation modules.

Reservation rules can be tested independently of the OODBMS.

The result is greater modularity, reuse, consistency, and easier persistence replacement.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

Legacy Transportation Management System

The legacy application has separate bus, train, taxi, and rental-vehicle modules, but construction logic is duplicated throughout the system. Client code knows concrete classes and constructor details, so introducing a new transport service requires changes in multiple places.

Common abstraction and relationships

Class	Key attributes	Responsibilities / methods
Transport	id, capacity, status, fare	start(), stop(), calculateFare(), getAvailability()
Bus	route, seatCount	start(), calculateFare()
Train	line, carriageCount	start(), calculateFare()
Taxi	driver, vehicleNumber	start(), calculateFare()
RentalVehicle	vehicleType, rentalRate	start(), calculateFare()
TransportFactory	configuration / provider context	createTransport()

Factory Method redesign

Define Transport as an interface or abstract class containing common operations.

Define a factory abstraction with createTransport(type/configuration).

Concrete factories such as BusFactory, TrainFactory, TaxiFactory, and RentalVehicleFactory create the correct product.

Clients depend on Transport and the factory abstraction rather than concrete constructors.

Use Abstract Factory when a complete family is required, such as Vehicle + TicketingPolicy + FarePolicy for a transport family.

Result

Creation logic becomes centralized and reusable. A new transport type is introduced by adding a product and factory rather than editing every client. Common transport behavior is represented once, improving extensibility, scalability, and code reuse.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

Legacy Healthcare Monitoring System

The healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform unrelated tasks and directly depend on concrete devices and storage components, creating low cohesion and strong coupling.

SOLID violations

Principle	Legacy violation	Redesign response
SRP	One class reads sensors, processes readings, stores records, alerts users, and reports.	Split responsibilities into sensor, processor, alert, repository, notification, and reporting components.
OCP	New sensor types or alert rules require edits to existing conditionals.	Use Sensor and AlertRule interfaces with new implementations.
DIP	High-level monitoring logic directly depends on concrete devices and storage.	Inject Sensor, Repository, Notification, and Rule abstractions.

Redesigned classes and interfaces

Abstraction / class	Key responsibility
Patient	patientId, demographics, care status; patient identity and domain data
Sensor	Interface: read(), getType()
ECGSensor / PulseOximeter / TemperatureSensor	Concrete sensor implementations
Reading	readingId, patientId, sensorType, value, unit, timestamp; immutable measurement
ReadingProcessor	validation and normalization of readings
AlertRule	Interface: evaluate(reading)
AlertService	creates/publishes alerts from rule results
MedicalRecordRepository	stores and retrieves readings and alerts
ReportGenerator	produces clinical or operational reports
MonitoringService	coordinates the monitoring use case through abstractions

Suitable patterns

Strategy — each AlertRule or analysis algorithm is a replaceable strategy.

Observer / Publisher-Subscriber — alert events can notify dashboards or clinical systems without coupling them to the processor.

Adapter — vendor-specific medical-device SDKs can be wrapped behind Sensor.

Dependency Injection — MonitoringService receives interfaces rather than constructing concrete infrastructure.

Result

The redesign improves cohesion because each component has a focused responsibility. OCP is supported because new sensors and rules can be added through implementations. DIP removes direct dependency on devices and storage. Mock sensors and repositories make testing easier, while adapters isolate vendor and infrastructure changes.

Cross-System Comparison

Quality	Legacy design	Redesigned design
Cohesion	Large classes mix unrelated concerns.	Focused entities, services, repositories, adapters, and UI components.
Coupling	Direct dependencies on concrete infrastructure and implementations.	Interfaces, dependency injection, and layered dependency direction.
Maintainability	Changes spread through conditionals and duplicated code.	Patterns isolate change points and responsibilities.
Extensibility	New features require edits to central modules.	New implementations can be added behind stable abstractions.
Testability	Tests require real database/devices/services.	Mocks, stubs, and in-memory repositories isolate business logic.
Scalability	Growth increases complexity inside existing modules.	Independent layers/components can evolve and scale separately.
Reuse	Queries and object creation are duplicated.	Repositories, DAOs, factories, services, and adapters are reusable.

Conclusion

The six redesigns convert tightly coupled legacy structures into modular object-oriented systems. Layered architecture separates presentation, application coordination, domain rules, and infrastructure. Strategy, Chain of Responsibility, and Observer isolate changing behavior; Factory Method and Abstract Factory centralize creation; Repository and DAO isolate persistence; Adapter protects the domain from external APIs; and Dependency Injection reverses dependency direction. Together, these changes produce systems that are easier to understand, test, extend, replace, and maintain.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation